

Introducción a REACT

Programación Web en el Entorno Cliente





Objetivos de Aprendizaje

- ❑ Comprender qué es React y por qué es importante
- ❑ Configurar un entorno de desarrollo con Create React App / Vite
- ❑ Entender JSX como sintaxis fundamental de React
- ❑ Crear y renderizar elementos básicos en React
- ❑ Desarrollar una primera aplicación "Portfolio"



Contenidos

- ❏ **Parte 1:** ¿Qué es React? Conceptos fundamentales
- ❏ **Parte 2:** Configuración con Create React App
- ❏ **Parte 3:** JSX básico y componentes funcionales
- ❏ **Práctica Final:** Primera aplicación React

¿QUE ES REACT?

- ❑ Es una biblioteca declarativa de JavaScript de código abierto diseñada para crear interfaces de usuario (UI).
- ❑ Fue creada en 2013 por Jordan Walker y actualmente es una de las bibliotecas de UI más populares.
- ❑ Su estructura se basa en dos conceptos fundamentales:



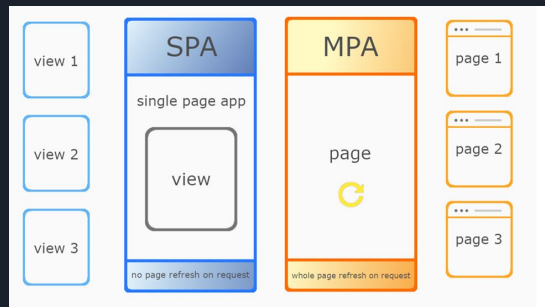
Componentes: Fragmentos de código reutilizables

Estados: Representan el estado de la información de la app a medida que el usuario interacciona con ella.

¿QUE ES REACT?

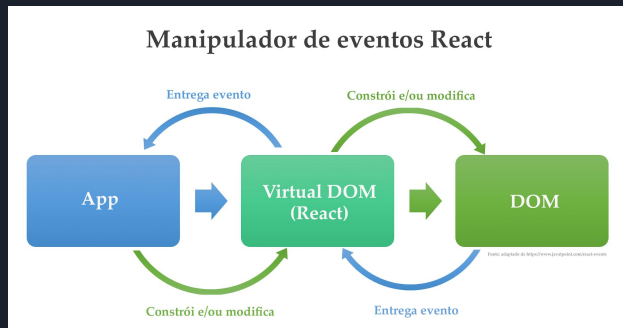
Conceptos

- **Single Page Applications (SPA)**



- **Virtual DOM:** representación del DOM real, que se almacena en la memoria del navegador, lo que permite que React realice actualizaciones de manera más rápida y eficiente.

Cuando modificas o creas algo, **React compara este árbol virtual con el DOM real del navegador** identificando los elementos que han cambiado y actualizando el DOM real de una forma eficiente y rápida,



¿Que es REACT?

React es una de las librerías más populares de JavaScript, debes tener claro que:

- React **no es un framework** sino que es una librería de JavaScript.  →

Tu como desarrollador tienes **el control** a la hora de estructurar el código

Un framework como Angular: te proporciona la estructura y te dice como y donde agregar tu código

- React es un proyecto de código abierto creado por Facebook.
- React se usa para construir interfaces de usuario (UI) en el frontend







Ventajas

Componentes reutilizables: "Como tener un bloque de lego ya construido para ensamblar en la arquitectura"

Virtual DOM: " Es la magia que hace React sea rápido"

Declarativo: "Le dices QUÉ quieres, no CÓMO hacerlo"

Comunidad: Hay infinidad de foros: Reddit, StackOverflow, <https://www.reactjs.wiki/>

Requisitos

- ❑ Estar familiarizado con HTML y CSS. ✓
Conocimientos básicos de JavaScript y programación. ✓
- ❑ Comprensión básica del DOM. ✓
- ❑ Estar familiarizado con la sintaxis y características de ES6. ✓
- ❑ Tener Node.js y npm instalados. ✓

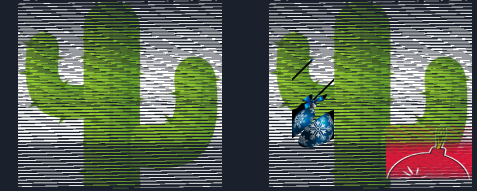
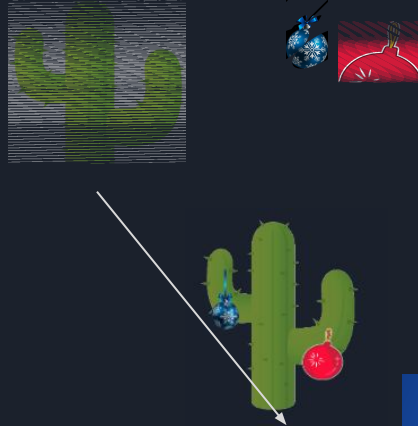
Virtual DOM

VIRTUAL DOM (LOCAL) → REACT

DOM REAL (NAVEGADOR)

Ciclo de vida

1. Cambias datos en React
- ↓
2. React crea NUEVO Virtual DOM
- ↓
3. React compara:
 - Virtual DOM anterior vs
 - Virtual DOM nuevo
- ↓
4. React identifica diferencias (diff)
- ↓
5. React actualiza SOLO esas partes en el DOM Real



React mantiene una copia 'virtual' de vuestra página en memoria. Cuando algo cambia, React compara la versión nueva con la anterior y solo actualiza las partes que realmente cambiaron.

REACT VS JAVASCRIPT TRADICIONAL

JavaScript

```
1 let contador = 0;
2 const boton = document.getElementById('boton');
3 const numero = document.getElementById('numero');
4
5 boton.addEventListener('click', () => {
6   contador++;
7   numero.textContent = contador;
8 });
```

REACT

```
1 function Contador() {
2   const [contador, setContador] = useState(0);
3
4   return (
5     <div>
6       <p>{contador}</p>
7       <button onClick={() => setContador(contador + 1)}>
8         Incrementar
9       </button>
10    </div>
11  );
12 }
```



56 K



Conceptos fundamentales

1. Componentes:

"Son como funciones de JavaScript,, devuelven pedazos de interfaz de usuario."



```
function MiComponente() {  
  return <h1>¡Hola!</h1>;  
}
```

2. JSX:

"Es HTML con [superpoderes](#). Se parece mucho a HTML pero permite incrustar la lógica de JS."



```
const elemento = <h1>¡Hola mundo!</h1>;
```

3. Props:

"Son como los parámetros de una función. Le pasáis datos a vuestros componentes."



```
<Saludo nombre="Ana" />
```

4. Estado:

"Son datos que pueden cambiar con el tiempo. Como los likes de una publicación o video."



```
const [nombre, setNombre] = useState("Ana");
```

Props y estados

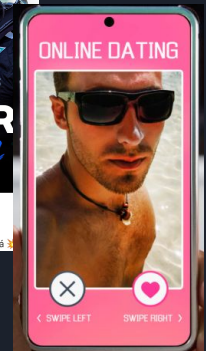
Características de los Props:

- ✓ **Solo lectura** - No puedes modificarlos
- ✓ **Vienen de arriba** - Los recibe del componente padre
- ✓ **Son permanentes** - No cambian dentro del componente

Características del Estado:

- ✓ Se puede modificar : Usando funciones como setMegusta
- ✓ Es interno : Cada componente maneja su propio estado
- ✓ Reactivo: Cuando cambia, el componente se re-renderiza.

```
1 function TarjetaUsuario(props) { // Props: nombre, foto
2   const [megusta, setMegusta] = useState(false); // Estado: si le gusta o no
3
4   return (
5     <div>
6       <img src={props.foto} />
7       <h3>{props.nombre}</h3> /* Props: no cambian */
8
9       <button onClick={() => setMegusta(!megusta)}>
10        {megusta ? "♥️" : "❤️"} /* Estado: cambia con clicks */
11      </button>
12    </div>
13  );
14 }
```





Ejemplos de uso de react

Sitios informativos simples: 1

- **Blogs personales** - WordPress, contenido estático
- **Páginas corporativas básicas** - "Quiénes somos", contacto
- **Portfolios de fotografía** - Galería simple de imágenes

Comercio & Negocios: 3

- **Airbnb** - Búsqueda, reservas, perfiles
- **Uber** - Panel de conductor y pasajero
- **Shopify** - Dashboard de tiendas
- **Dropbox** - Interfaz web de archivos

Redes Sociales & Entretenimiento: 2

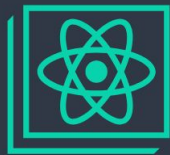
- **Facebook** - Creadores de React, toda la plataforma
- **Instagram** - Interfaz web completa
- **Netflix** - Panel de usuario, navegación de contenido
- **WhatsApp Web** - Chat en tiempo real
- **Discord** - Interfaz de chat y servidores

Sitios gubernamentales/institucionales: 4

- **Páginas de universidades** - Información de carreras, horarios
- **Sitios gubernamentales** - Solo información pública

PARTE 2 CONFIGURACIÓN DEL ENTORNO

Configurar un entorno de desarrollo con Create React App / Vite



Create React App

<https://create-react-app.dev/>



<https://vite.dev/>

¿QUÉ ES VITE?

Vite (pronunciado "vit", como "rápido" en francés) es una **herramienta de construcción** moderna para aplicaciones web que hace el desarrollo **extremadamente rápido**.

¿Qué tipo de herramienta es?

- **Build Tool** (herramienta de construcción)
- **Dev Server** (servidor de desarrollo)
- **Bundler** (empaquetador de código)
- **Todo-en-uno** para desarrollo web moderno

¿Para qué sirve?

-  Configurar proyectos React automáticamente
-  Servir tu aplicación en desarrollo
-  Recargar cambios instantáneamente
-  Empaquetar tu app para producción
-  Optimizar el código automáticamente

En resumen:

Nos ayuda
crear un
proyecto base



Requisitos para el entorno Vite

1. **Node.js** (versión 18 o superior)  ¡Nota: Vite requiere versión más reciente que CRA!
 - Descargar desde: <https://nodejs.org>
 - Verificar instalación: `node --version`
 - **Mínimo requerido: v18.0.0**
 -
2. **npm** (incluido con Node.js) o **pnpm/yarn**
 - Verificar instalación: `npm --version`
 -
3. **Editor de código** (VS Code)

Verificación rápida:

bash

```
node --version # Debe mostrar v18+
```

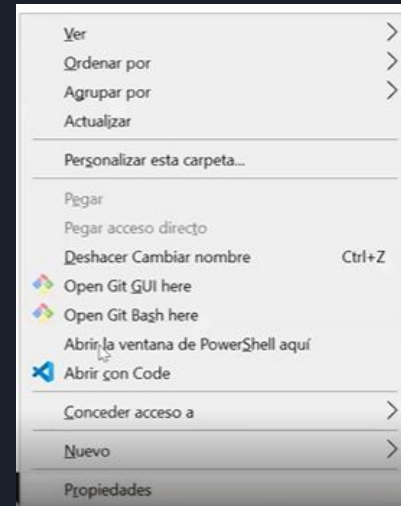
```
npm --version # Debe mostrar 8+
```


Creación del proyecto

1. Habilitar ejecución de scripts en powerShell → Como administrador

`Set-ExecutionPolicy Unrestricted` y le decimos que si (S)

2. Nos situamos en una carpeta donde queremos crear el proyecto
3. Abrimos la powerShell de windows en esa carpeta Ctrl+Shift+Click Derecho
4. Una vez posicionados ejecutamos el cmd → `npm create vite@latest`
5. Seguimos la config de Vite: Nombre, React, JavaScript.
6. Abrimos el VisualCode en la carpeta que se nos ha creado y en la terminal de bash: `npm install` y despues `npm run dev`



```
vite.C:\Users\Usuario\Documents\HB\JSB44\
|
|  Project name:
|  prueba
|
|  Select a framework:
|  React
|
|  Select a variant:
|  JavaScript + SWC
|
|  Scaffolding project in C:\Users\
|
|  Done. Now run:
|
|  cd prueba
|  npm install
|  npm run dev
```

Estructura del proyecto

```
mi-primer-app/  
├── public/  
│   └── vite.svg      ← Logo de Vite  
├── src/  
│   ├── App.jsx      ← Componente principal (.jsx!)  
│   ├── App.css       ← Estilos del componente principal  
│   ├── main.jsx      ← Punto de entrada (era index.js)  
│   └── index.css     ← Estilos globales  
├── index.html        ← ¡HTML en la raíz! (diferencia importante)  
├── package.json      ← Configuración y dependencias  
├── vite.config.js     ← Configuración de Vite  
└── README.md         ← Documentación del proyecto
```

```
> node_modules  
> public  
✓ src  
  > assets  
    App.css  
    App.jsx  
    index.css  
    main.jsx  
  .gitignore  
  eslint.config.js  
  index.html  
  package-lock.json  
  package.json  
  README.md  
  vite.config.js
```

Flujo de una aplicación de React +Vite

Diagrama visual:

<https://claude.ai/public/artifacts/c2fa27ac-9973-486d-a9a9-f87845bb823>



Puntos importantes:

- **Solo hay 1 HTML** - el resto son componentes React
- **main.jsx es el puente** entre HTML tradicional y React
- **App.jsx es donde programas** tu aplicación real
- **El flujo es unidireccional** - siempre va en el mismo orden



Flujo de una Aplicación React + Vite



index.html

Punto de entrada - La puerta principal

```
<div id="root"></div> <script src="/src/main.jsx"></script>
```

¿Qué hace? Crea un contenedor vacío (#root) y carga main.jsx



Navegador carga main.jsx



main.jsx

Inicializador de React - El motor

```
import App from './App.jsx'
createRoot(document.getElementById('root')).render(<App />)
```

¿Qué hace? Busca el div #root e inyecta el componente <App />



Comandos esenciales

Comandos principales de npm:

bash

Iniciar servidor de desarrollo

`npm run dev`

↳ Abre la app en http://localhost:5173

↳ Recarga INSTANTÁNEAMENTE cuando guardas cambios

Construir para producción

`npm run build`

↳ Crea versión optimizada en carpeta 'dist'

Vista previa del build de producción

`npm run preview`

↳ Sirve la versión de producción localmente

Parar el servidor

Ctrl + C (en la terminal)

Para desarrollo usaremos principalmente:

- `npm run dev` - Para desarrollo **Ctrl + C** - Para parar



Introducción a JSX

JSX → HTML dentro de JavaScript

- Es una extensión de sintaxis JavaScript
- Permite escribir estructuras similares a HTML dentro de JavaScript.
- Se "transpila" (convierte) a JavaScript normal. (Babel)
- Define la estructura de un componente de React.

```
// Esto es JSX
const elemento = <h1>¡Hola mundo!</h1>;

// Se convierte en esto (internamente por Babel/ESBuild):
const elemento = React.createElement(
  'h1',
  null,
  '¡Hola mundo!'
);
```

Sintaxis básica de JSX

Reglas básicas

1. Cada componente debe tener un contenedor raíz / padre. (div, header, ..) en el que almacenaremos los elementos hijos.

¿Que pasa si queremos agregar más de un contenedor? Utilizamos un contenedor invisible para el DOM que engloba a todos los demás. `<> otros contenedores </>`

FRAGMENT

2. Las etiquetas deben cerrarse
3. Usar className en lugar de class.

4. Los componentes(funciones) deben ir en mayúscula

```
function App() {  
  const [count, setCount] = useState(0)
```

```
// ✅ Correcto  
return (  
  <div>  
    <h1>Título</h1>  
    <p>Párrafo</p>  
  </div>  
);
```

```
// ❌ Incorrecto  
return (  
  <h1>Título</h1>  
  <p>Párrafo</p>  
);
```

```
// ✅ Correcto  
  
<br />  
  
// ❌ Incorrecto  
  
<br>
```

```
<div className="mi-clase">Contenido</div>
```



Sintaxis básica de JSX

Expresiones

- Usando llaves para expresiones JavaScript { }

```
function Saludo() {  
  const nombre = "Ana";  
  const edad = 25;  
  
  return (  
    <div>  
      <h1>¡Hola {nombre}!</h1>  
      <p>Tienes {edad} años</p>  
      <p>El próximo año tendrás {edad + 1} años</p>  
      <p>Es {edad} >= 18 ? 'mayor' : 'menor' de edad</p>  
    </div>  
  );  
}
```

¿Qué puedes poner entre llaves?

- Variables: {nombre}
- Operaciones: {2 + 3}
- Llamadas a funciones: {obtenerFecha()}
- Expresiones condicionales: {edad >= 18 ? 'Adulto' : 'Joven'}



Mi primer Componente

¿Qué es un componente?

Una función que retorna JSX - es como una función que crea elementos HTML.

Representan piezas reutilizables de interfaz de usuario.

```
function MiComponente() {  
  return <h1>¡Soy un componente!</h1>;  
}  
  
// También se puede escribir como arrow function:  
const MiComponente = () => {  
  return <h1>¡Soy un componente!</h1>;  
}
```

Reglas:

- El nombre debe empezar con mayúscula.
- Debe retornar JSX.
- Solo puede retornar un elemento raíz.

Usando componentes

Escribes una vez, utilizas muchas veces

Main.jsx

```
1 function Saludo () {
2   return(
3     <>
4       <head>Bienvenido a mi app</head>
5       <div>
6         <h1>Hola mi nombre es JOSE</h1>
7       </div>
8     </>
9   )
10 }
11
12 export default Saludo;
```

App.jsx

```
1 import Saludo from './Saludo'
2 import './App.css'
3
4 function App() {
5   return (
6     <>
7       <Saludo />
8       <Saludo />
9       <Saludo />
10    </>
11  )
12 }
13 export default App
14
```



Hola mi nombre es JOSE

Hola mi nombre es JOSE

Hola mi nombre es JOSE

Renderizado

Conectando React con el HTML (en Vite)

El flujo:

El navegador carga index.html.

El `<script type="module" src="/src/main.jsx"></script>` carga el punto de entrada de React.

React busca el elemento con `id="root"`.

`ReactDOM.createRoot().render()` renderiza el componente App dentro de ese elemento.

App puede contener otros componentes.

Todo se muestra en la página.



```
1 <body>
2   <div id="root"></div>
3   <script type="module" src="/src/main.jsx"></script>
4 </body>
```



```
1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import './index.css'
4 import App from './App.jsx'
5
6 createRoot(document.getElementById('root')).render(
7   <StrictMode>
8     <App />
9   </StrictMode>,
10 )
```

Ejercicio Práctico

```
1  const content = () => {
2    return(
3      <>
4        <h1>Bienvenidos a mi primera pagina</h1>
5        <h2>Estamos creando nuestro primer componente con React + Vite</h2>
6        <article>
7          <p className="text">Lorem ipsum, dolor sit amet consectetur adipisicing elit. Harum rat
8          
27     <h2>Mis hobbies</h2>
28     <p>Cuando era pequeño me gustaba mucho jugar al {hobby1} con mi padre. Hoy en día me gusta mas {hobby2}
29   </p>
30   </>
31 )
32 }
```



Proyecto Integrador

Tarjeta de presentación

Objetivo: Crear una tarjeta de presentación personal usando React

Requisitos:

Componente principal App

Componente **Cabecera** con tu nombre y título.

Componente **Información** con datos personales.

Componente **Habilidades** con tus habilidades/intereses.

Usar variables JavaScript para la información.

Aplicar estilos CSS básicos

```
1  function App() {
2      return (
3          <div className='tarjeta-presentacion'>
4              <Cabecera />
5              <Informacion />
6              <Habilidades />
7          </div>
8      )
9  }
10 export default App
```



```
function Cabecera() {
  const nombre = "Tu Nombre";
  const titulo = "Estudiante de Desarrollo Web";

  return (
    <header>
      <h1>{nombre}</h1>
      <h2>{titulo}</h2>
    </header>
  );
}

function Informacion() {
  // Tu código aquí
  return (
    <section>
      {/* Información personal */}
    </section>
  );
}

function Habilidades() {
  // Tu código aquí
  return (
    <section>
      {/* Tus habilidades */}
    </section>
  );
}
```



Posibles estilos

Podéis utilizarlos en el App.css

La Tarjeta-presentación está asignado al contenedor padre de la función App.

```
1  /* App.css */
2  .Tarjeta-Presentacion {
3      max-width: 600px;
4      margin: 50px auto;
5      padding: 30px;
6      border: 2px solid #61dafb;
7      border-radius: 10px;
8      font-family: Arial, sans-serif;
9      background-color: #f9f9f9;
10 }
11 header {
12     text-align: center;
13     border-bottom: 1px solid #ccc;
14     padding-bottom: 20px;
15     margin-bottom: 20px;
16 }
17
18 header h1 {
19     color: #61dafb;
20     margin: 0;
21 }
22
23 header h2 {
24     color: #666;
25     margin: 5px 0 0 0;
26     font-weight: normal;
27 }
28
29 section {
30     margin: 20px 0;
31 }
```



Recurso adicionales

Para seguir aprendiendo

Documentación oficial:

- [Documentación de React](#)
- [Tutorial oficial de React](#)
- [Documentación oficial de Vite](#)

Herramientas útiles:

- [React Developer Tools](#) - Extensión para Chrome
- [CodePen](#) - Para experimentar con React online
- [CodeSandbox](#) - IDE online para React



Resumen de la sesión

Conceptos clave de React:

- **React** es una biblioteca para crear interfaces de usuario.
- **Vite** nos ayuda a configurar proyectos rápidamente y ofrece un desarrollo ultra-rápido.
- **JSX** permite escribir HTML dentro de JavaScript.
- **Componentes** son funciones que retornan JSX.
- **Virtual DOM** optimiza las actualizaciones.

Habilidades prácticas:

- Crear proyecto React con `npm create vite@latest`.
- Escribir componentes funcionales básicos.
- Usar expresiones JavaScript en JSX.
- Organizar código en múltiples componente

